The Coupon Collector Problem

Computational Problems involving Marko Chains

Dr Kieran Murphy

SETU (Waterford)

Autumn Semester, 2026/27

Introduction

- What is the coupon collector problem?
- Who is to blame?
- What applications does it have?

The Coupon Collector Problem

A company decided to include a toy in their cereal boxes.



What is the expected number of boxes purchased in order to obtain all of the toys?

Some variations …

Cards collected as packs

Trading cards are obtained in packs of a fixed size.



Typically no repetition within a pack.

Unequal probabilities

Not all cards are equally likely.



A lot of effort is put into tuning the probabilities to maximise impact (increase demand) or minimise costs (prizes)

Multiple collections

One collector might want multiple collections



or multiple collectors working together, trading cards, so that all get a full collection.

The Coupon Collector Problem — Specification

- Number of distinct **coupons** (trading cards, coins, etc.) is $n > 0$.
- Assumptions:
  - Coupons are obtained one at a time.
    - Later we will consider **packs** of size k with no repetition within a pack.
  - The number of copies of each coupon is effectively infinite.
    - If the number of copies of each coupon was small enough then the probabilities would change during the experiment based on which coupon have been seen already. *(so would have a sampling without replacement problem — harder).*
    - Note: "effectively infinite" does not mean the actual number is very big, just that it is big enough.
  - Each coupon is equally likely to be found, i.e., uniform probabilities
    - uniform distribution — easiest but unrealistic for most real-life situations.
    - **Zipf–Mandelbrot** distribution — more realistic (**power-law**) distribution.

What is the expected number of coupons collected in order to obtain m complete collections of the coupons?

Aside — History of the coupon collector problem

1708 First appeared in *De Mensura Sortis (On the Measurement of Chance)* by A. De Moivre.

- Additional results by various authors including Laplace and Euler in the case of uniform probabilities, i.e., when $p_j = \frac{1}{n}$ for all j.

1954 H. Von Schelling obtained waiting time to complete a collection for non-uniform probabilities.

1960 D. J. Newman and L. Shepp calculated waiting time for two collections ($m = 2$).

Applications

- Electrical engineering — related to the cache fault problem, also in electrical fault detection.
- Biology — used to estimate the number of species of animals (see Watterson estimator).
- Physics — how long before at least one gas molecule, from an opened capsule, has visited every discrete macro-region of the room?
- Medicine — how many random individuals in a mixed population must encounter an antigen (or get vaccinated) before every sub-community or demographic pocket has attained immunity.

Simulation

- **Contrapositive of “If we don’t understand it, then we can’t code it”.**

First a simulation ...

Before we construct a Markov chain model lets code a simulation ... first, code snippets ...

```
1   rng = default_rng(42)          # fixed seed during testing
2
3   n = 4
4   space = range(n)               # all possible coupons
5   collected = set()              # coupons collected to date
6   found = None                   # coupons found in current trial
7   count = 0
8
9   print (f'{count = :3d}  \t{found = }  \t{collected = }')
10  while len(collected)<n:  # collection is incomplete
11
12      found = int(rng.choice(space))    # get next (random) coupon
13      collected |= {found}              # use sets (union) so duplicates dropped
14      count += 1
15
16      print (f'{count = :3d} \t{found = } \t{collected = }')
```

… wrap code up in a function …

```
1   def run_experiment(n, seed=None, debug=False):
2
3       rng = default_rng(seed)
4
5       space = range(n)          # all possible coupons
6       collected = set()         # coupons collected to date
7       found = None              # coupons found in current trial
8
9       count = 0
10      if debug: print (f'{count = :3d} \t{found = } \t{collected = }')
11      while len(collected)<n:  # not completed collection yet
12
13          found = int(rng.choice(space))    # get next (random) coupon
14          collected |= {found}              # use sets (union) so duplicates dropped
15          count += 1
16
17          if debug: print (f'{count = :3d} \t{found = } \t{collected = }')
18
19      return count
```

Using optional parameters we can set the seed for reproducible results and displaying debug output.

… and a few sample runs …

```
run_experiment(5, seed=105, debug=True)
```

```
count =   0   found = None  collected = set()
count =   1   found = 1   collected = {1}
count =   2   found = 3   collected = {1, 3}
count =   3   found = 4   collected = {1, 3, 4}
count =   4   found = 4   collected = {1, 3, 4}
count =   5   found = 3   collected = {1, 3, 4}
count =   6   found = 0   collected = {0, 1, 3, 4}
count =   7   found = 3   collected = {0, 1, 3, 4}
count =   8   found = 2   collected = {0, 1, 2, 3, 4}
8
```

```
run_experiment(5, seed=1013, debug=True)
```

```
count =   0   found = None  collected = set()
count =   1   found = 0   collected = {0}
count =   2   found = 3   collected = {0, 3}
count =   3   found = 2   collected = {0, 2, 3}
count =   4   found = 0   collected = {0, 2, 3}
count =   5   found = 4   collected = {0, 2, 3, 4}
count =   6   found = 3   collected = {0, 2, 3, 4}
count =   7   found = 4   collected = {0, 2, 3, 4}
count =   8   found = 2   collected = {0, 2, 3, 4}
count =   9   found = 3   collected = {0, 2, 3, 4}
count =  10   found = 3   collected = {0, 2, 3, 4}
count =  11   found = 3   collected = {0, 2, 3, 4}
count =  12   found = 1   collected = {0, 1, 2, 3, 4}
12
```

```
run_experiment(3, seed=2, debug=True)
```

```
count =   0   found = None  collected = set()
count =   1   found = 2   collected = {2}
count =   2   found = 0   collected = {0, 2}
count =   3   found = 0   collected = {0, 2}
count =   4   found = 0   collected = {0, 2}
count =   5   found = 1   collected = {0, 1, 2}
5
```

Need to get some idea of variation … so repeat runs …


```
import scipy.stats as stats

data = [run_experiment(4) for _ in range(10)]
print(data)
m, se = np.mean(data), stats.sem(data)
print(f"\n95% CI for number of coupons = {m:.2f} ± {1.96*se:.2f}" )
```

```
[10, 5, 8, 15, 13, 8, 4, 6, 7, 8]

95% CI for number of coupons = 8.40 ± 2.13
```

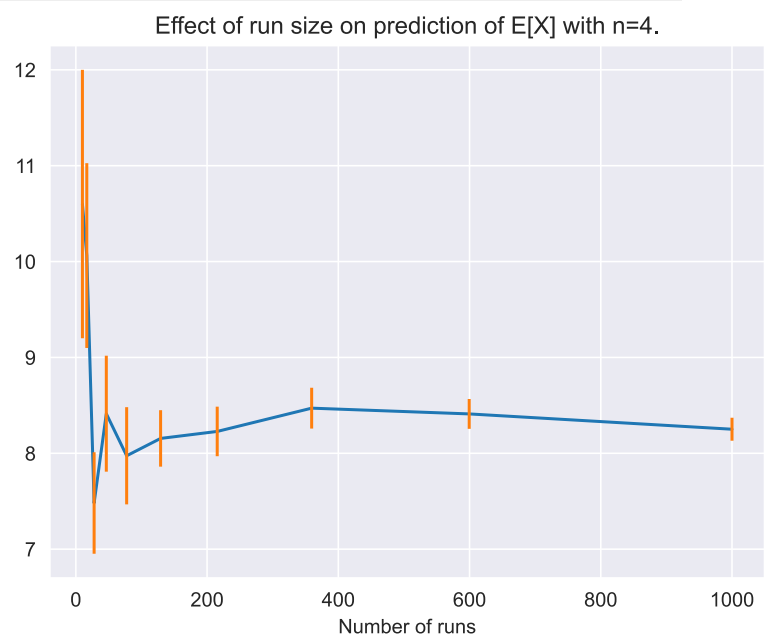
```
data = [run_experiment(4) for _ in range(100)]
m, se = np.mean(data), stats.sem(data)
print("\n95%% CI for number of coupons = %s ± %.2f" % (m, 1.96*se) )
```

```

95% CI for number of coupons = 8.57 ± 0.82
```

… a picture is worth a thousand words …

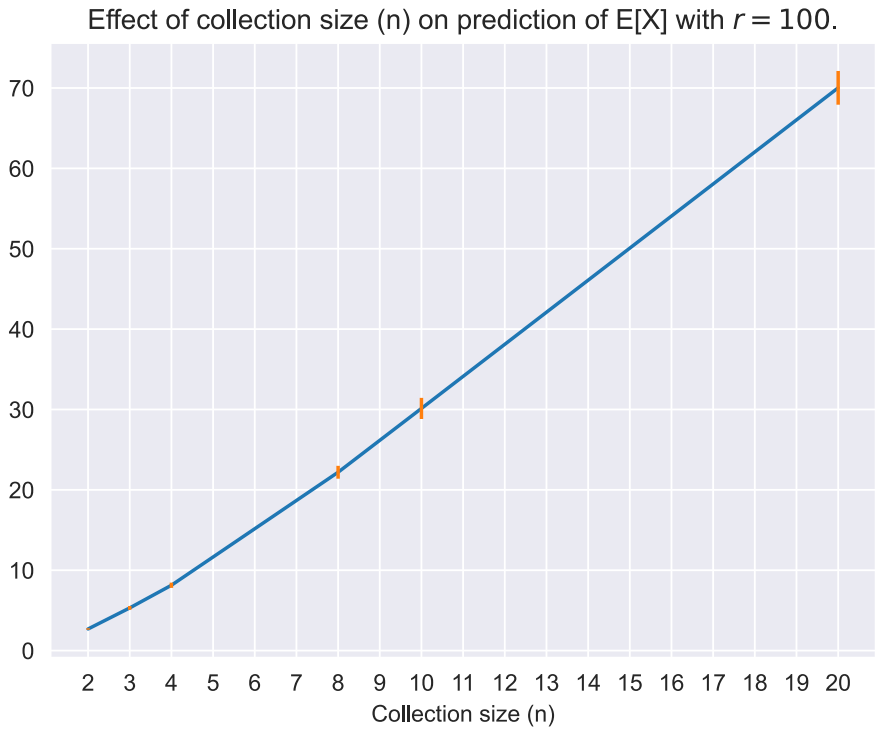

```
n = 4
r_list = np.logspace(1,3,10)
m_list = []
se_list = []
for r in r_list:
    data = [run_experiment(n) for _ in range(int(r))]
    m_list.append(np.mean(data))
    se_list.append(stats.sem(data))
```



- Confidence intervals are shrinking ✅
- But only beginning to overlap — prob need larger samples
- Best estimate for expected value is about 8.5

```
plt.plot(r_list, m_list)
plt.errorbar(r_list, m_list, se_list, linestyle='None')
plt.xlabel("Number of runs")
plt.title(f"Effect of run size on prediction of E[X] with {n=}.")
plt.show()
```

Effect of collection size (n) ...




```
 n  mean   se
 2  2.69 0.10
 3  5.31 0.23
 4  8.13 0.33
 8 22.19 0.80
10 30.13 1.31
20 70.02 2.10
```

Variance increases with collection size (n), but this is offset by the fact that the estimate for the expected number of coupons needed is increasing faster.

```
n_list = [2,3,4,8,10,20]
m_list = []
se_list = []
for n in n_list:
    data = [run_experiment(n) for _ in range(100)]
    m_list.append(np.mean(data))
    se_list.append(stats.sem(data))
```

Theory

- **We can also calculate the expected number of coupons needed using a theoretical approach. It is exact but less flexible than the simulation approach.**

Theoretical approach — via geometric distribution

Let X denote the (random) number of coupons that we need to purchase in order to complete our collection of n coupons. Then X can be expressed as the sum

$$\underbrace{X}_{\substack{\text{number of coupons needed to collect all } n \text{ coupons}}} = \underbrace{X_0}_{\substack{\text{number of coupons needed to go from 0 to 1 coupon collected}}} + \underbrace{X_1}_{\substack{\text{number of coupons needed to go from 1 to 2 coupons collected}}} + \cdots + \underbrace{X_i}_{\substack{\text{number of coupons needed to go from } i \text{ to } i+1 \text{ coupons collected}}} + \cdots + \underbrace{X_{n-1}}_{\substack{\text{number of coupons needed to go from } n-1 \text{ to } n \text{ coupons collected}}}$$

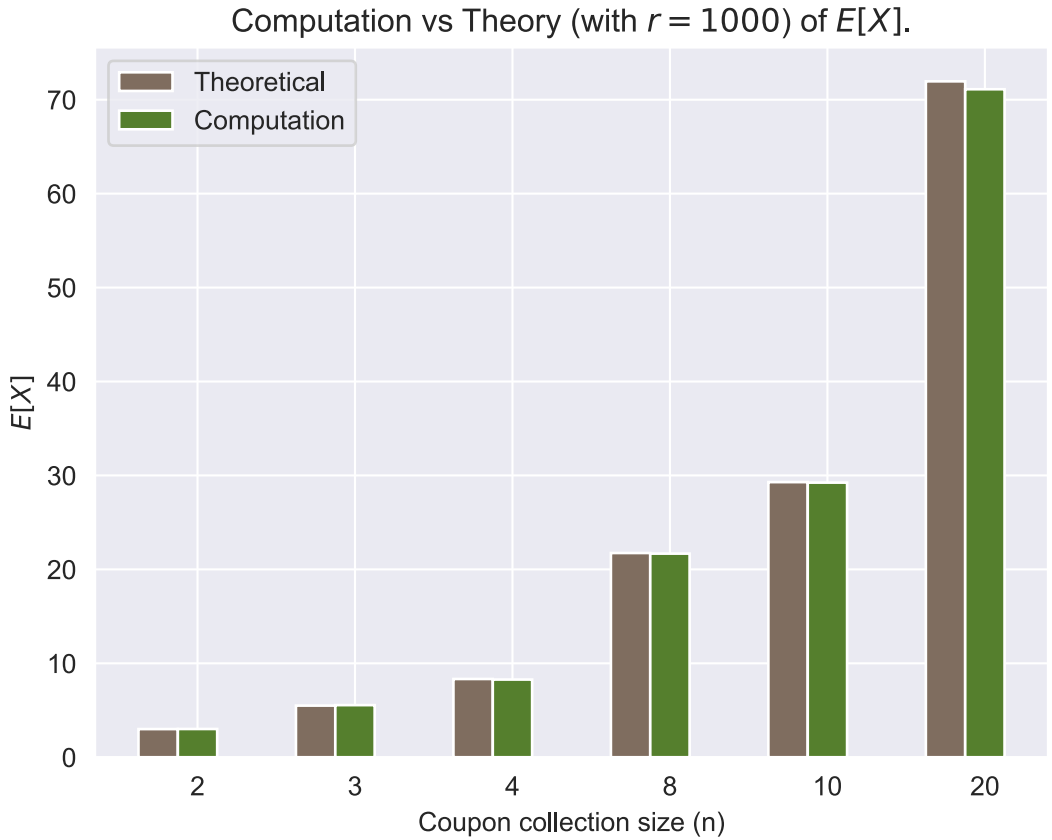
$$X_0 \sim \mathrm{GP}(1) \qquad X_1 \sim \mathrm{GP}\left(\tfrac{n-1}{n}\right) \qquad X_i \sim \mathrm{GP}\left(\tfrac{n-i}{n}\right) \qquad X_{n-1} \sim \mathrm{GP}\left(\tfrac{1}{n}\right)$$

Applying expectation and using linearity of expectation we have

$$\begin{aligned} E[X] &= E[X_0] + E[X_1] + E[X_2] + \cdots + E[X_{n-1}] \\ &= \frac{n}{n} + \frac{n}{n-1} + \frac{n}{n-2} + \cdots + \frac{n}{1} = n \sum_{i=1}^{n} \frac{1}{i} = nH_n \end{aligned}$$

where $H_n = \sum_{i=1}^{n} \frac{1}{i}$ is the **n-th harmonic number**. (For large n the harmonic number grows logarithmically, $H_n \approx \ln n + \gamma$, where $\gamma \approx 0.5772\ldots$ is the Euler-Mascheroni constant).

Comparison of Computation vs Theory of $E[X]$

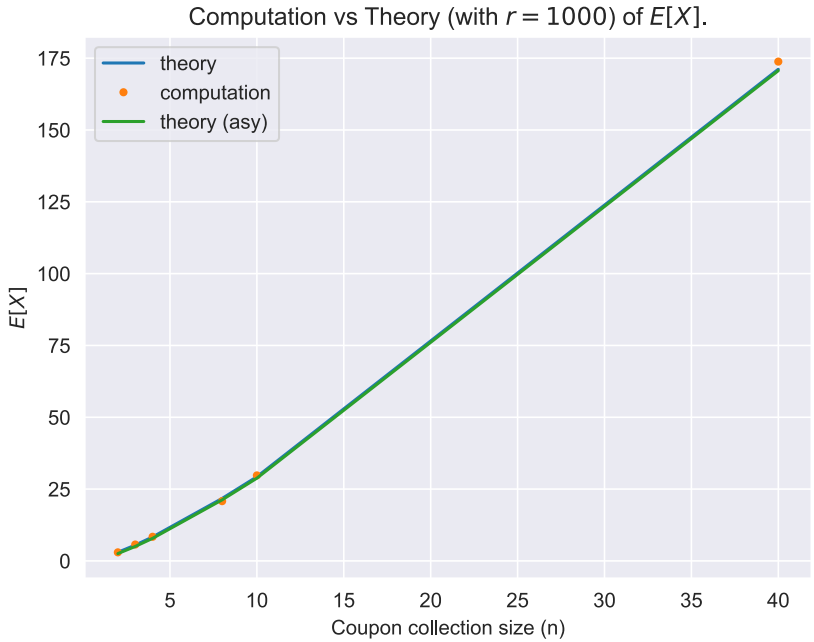


n	Theoretical	Computation
2	3.000000	3.010
3	5.500000	5.550
4	8.333333	8.272
8	21.742857	21.679
10	29.289683	29.237
20	71.954793	71.113

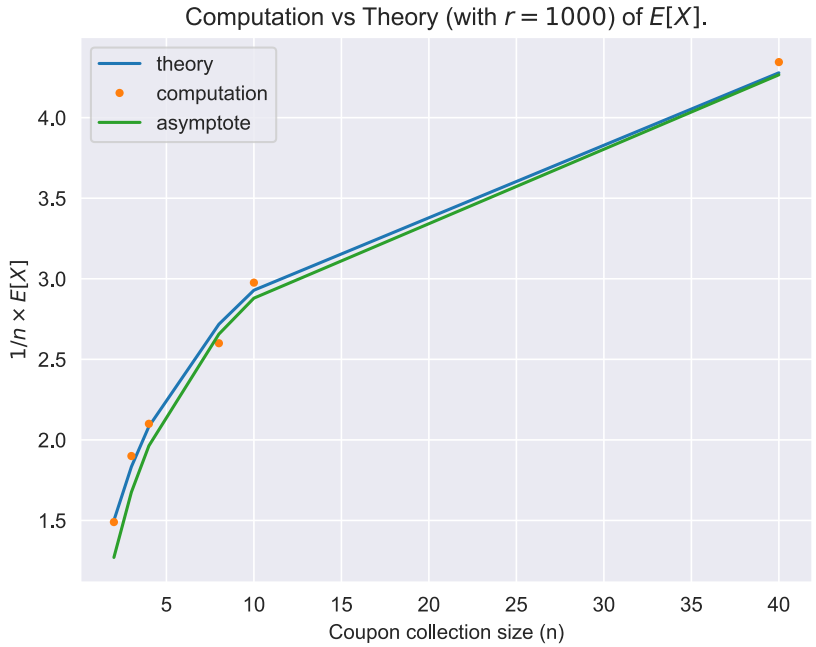
- Theoretical and computation agree.
- The computation result is less precise but it is much easier to apply to extensions to the basic problem.

Comparison of Computation vs Theory vs Asymptote of $E[X]$

The theoretical value has asymptotic $n(\ln n + \gamma)$. Plotting that with exact theory expression for $E[X_n]$ and values computed using `run_experiment` we have



Graph of $E[X_n]$ is linear-log so is dominated by the n



In graph of $E[X_n]/n$ it is easier to see the non-linear component.

Markov Chain Model

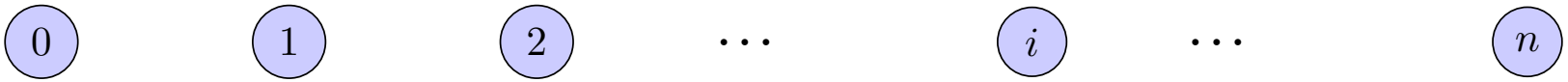
- Model the problem as a Markov chain.
- Big issue here is when is this better/worse then the simulation approach.

Markov Chain Model

Model

- **State**: $i, i = 0, l..., n$, where i is number of collected coupons.

- Initial state is 0. State n is terminal.
- **Stage**: Number of coupons purchased.



Markov Chain Model

Model

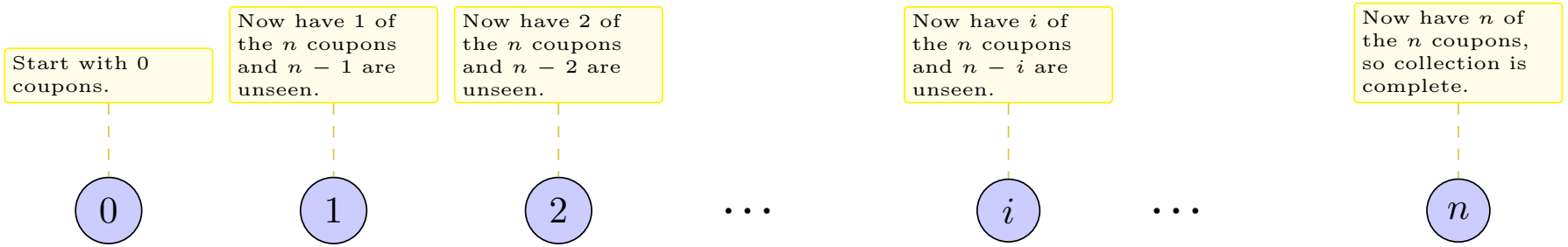
- **State**: i, $i = 0, l..., n$, where i is number of collected coupons.

  We have i of the n available coupons, so $(n - i)$ coupons are unseen.

- Initial state is 0. State n is terminal.

- **Stage**: Number of coupons purchased.

  How long does it take to travel from 0 to n?



Markov Chain Model

Model

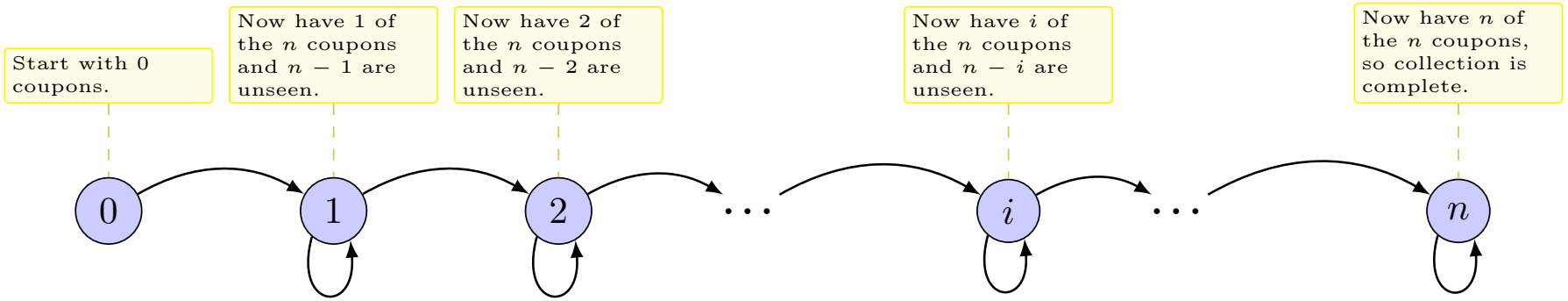
- **State**: $i, i = 0, l..., n$, where i is number of collected coupons.

  We have i of the n available coupons, so $(n - i)$ coupons are unseen.

- Initial state is 0. State n is terminal.

- **Stage**: Number of coupons purchased.

  How long does it take to travel from 0 to n?



Markov Chain Model

Model

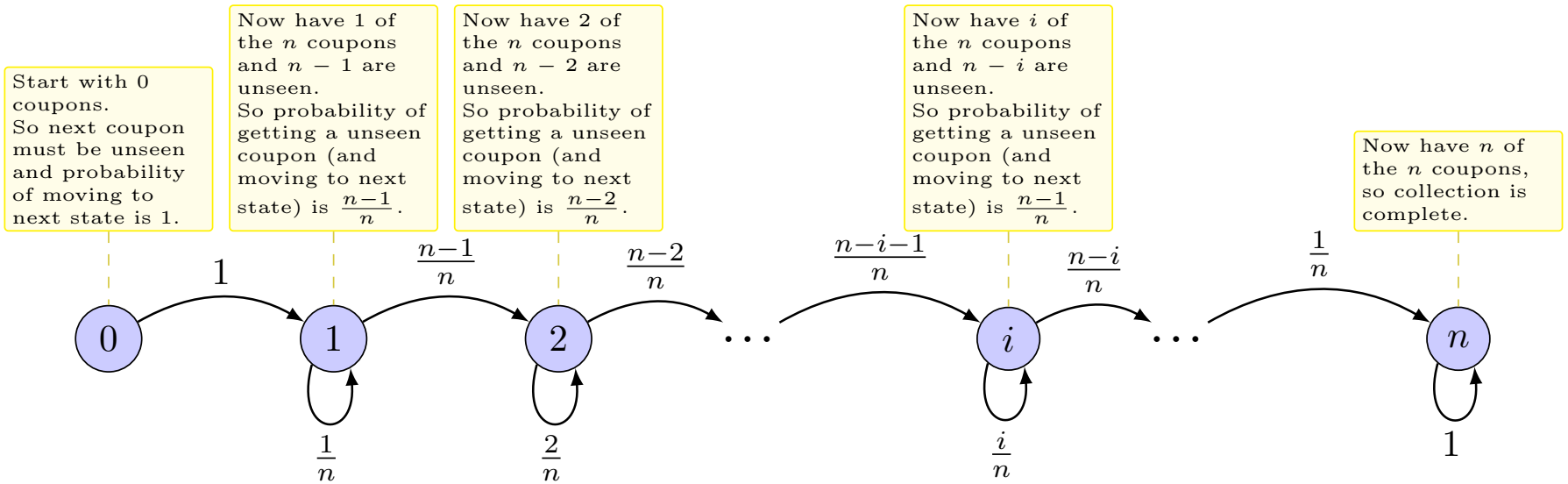
- **State**: i, $i = 0, l..., n$, where i is number of collected coupons.

  We have i of the n available coupons, so $(n - i)$ coupons are unseen.

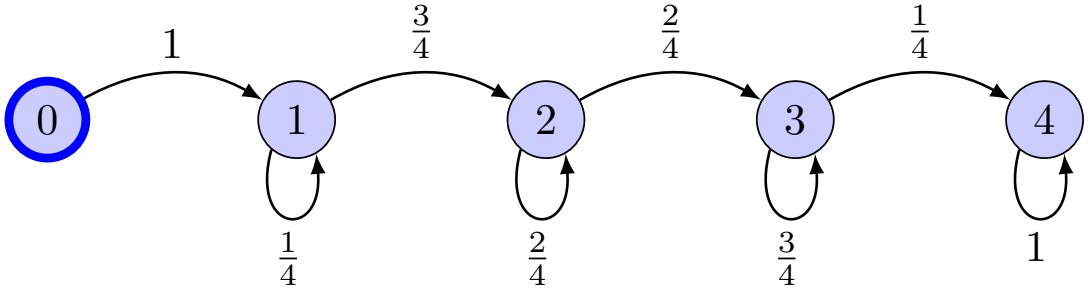
- Initial state is 0. State n is terminal.

- **Stage**: Number of coupons purchased.

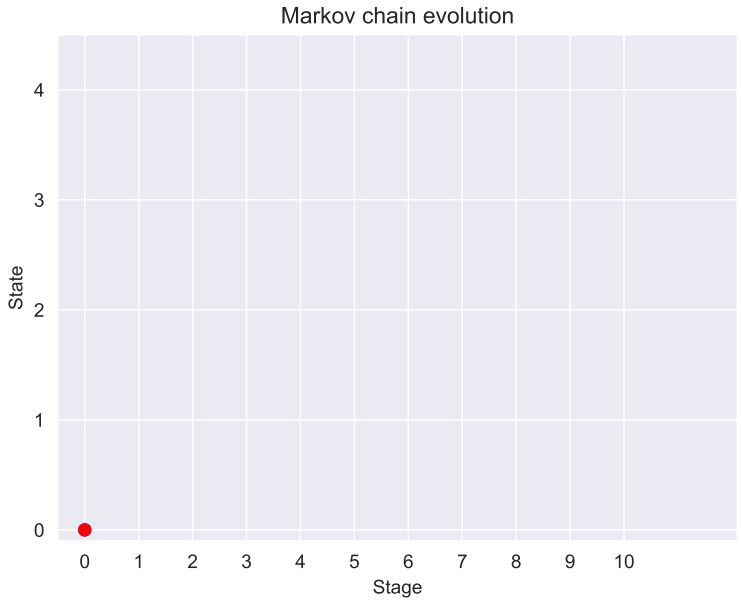
  How long does it take to travel from 0 to n?



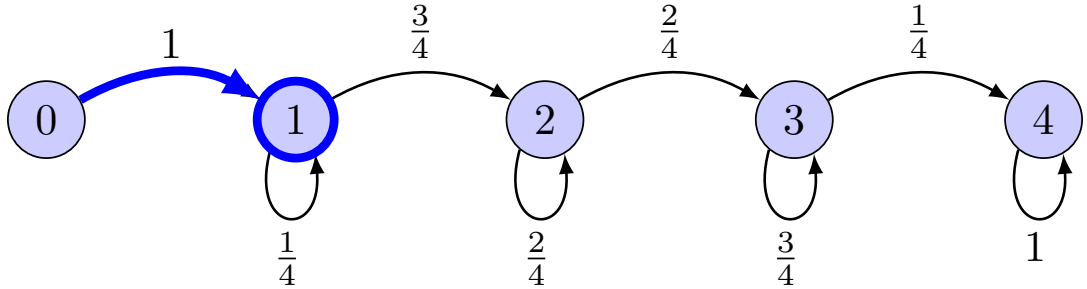
Viewing our first simulation run as a Markov chain …



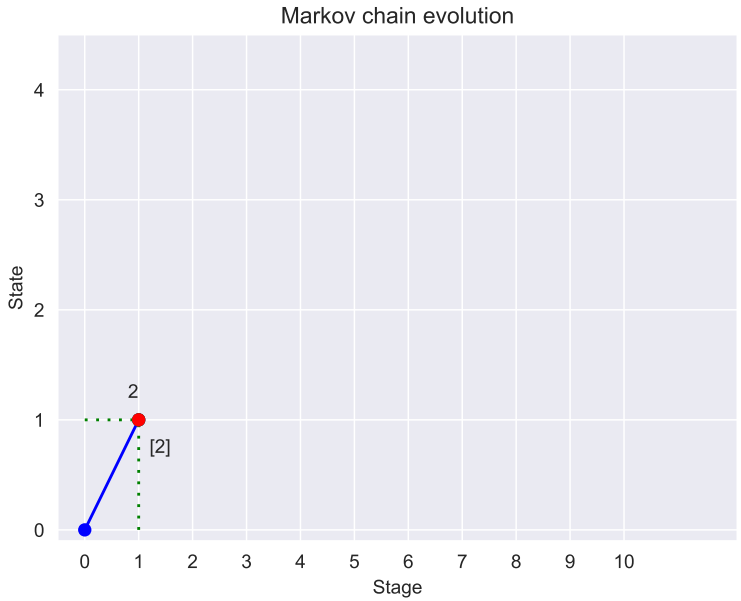

```
stage =   0   found = None  collected = set()
```



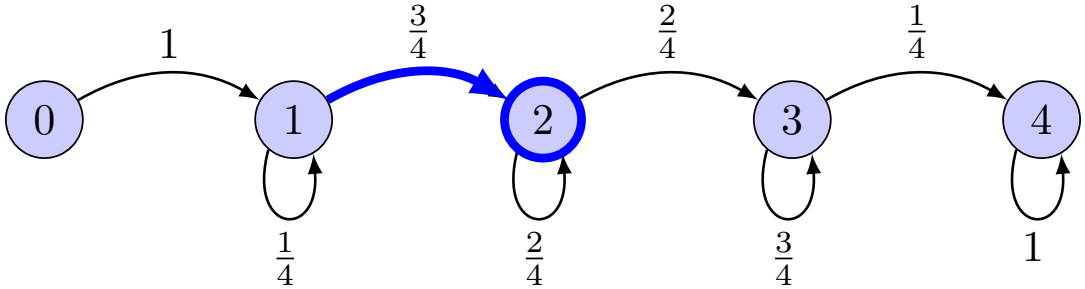
Viewing our first simulation run as a Markov chain …



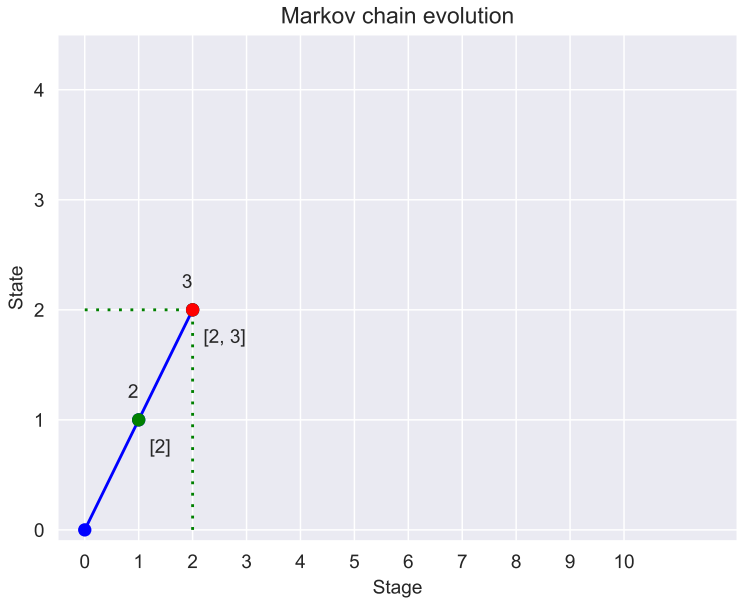

```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
```



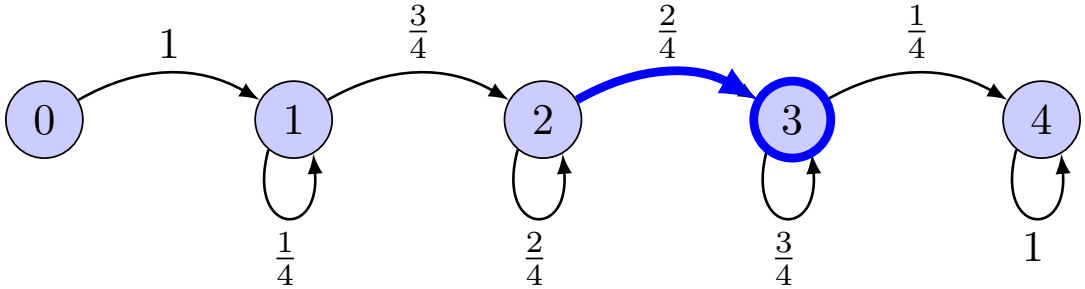
Viewing our first simulation run as a Markov chain …



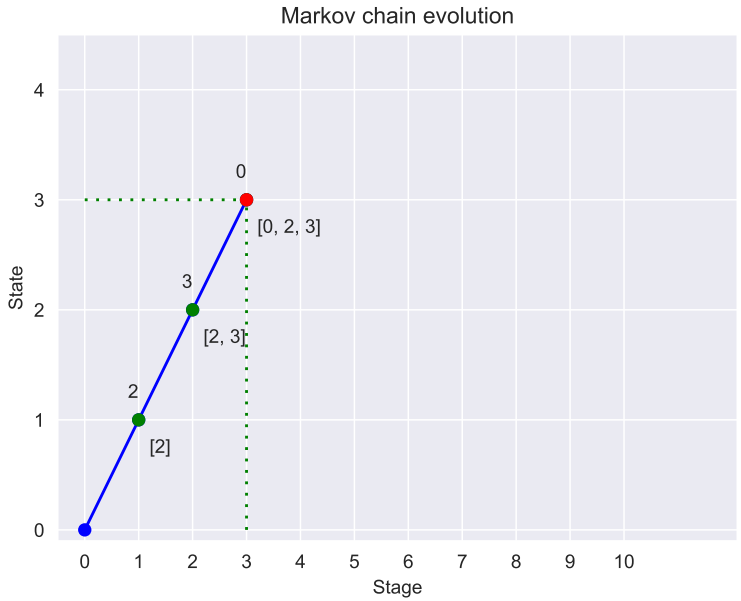

```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
```



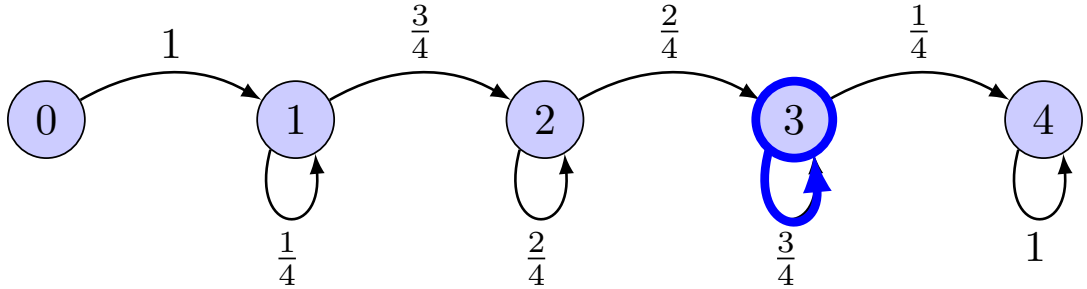
Viewing our first simulation run as a Markov chain …



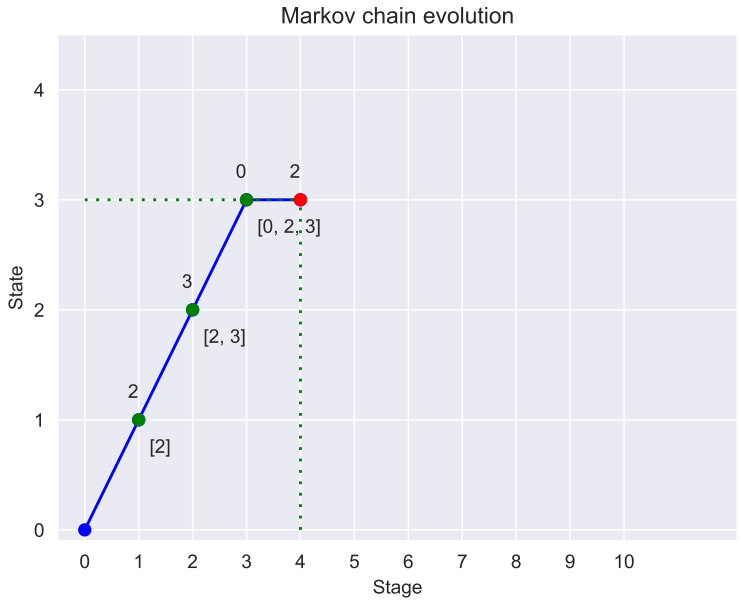

```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
```



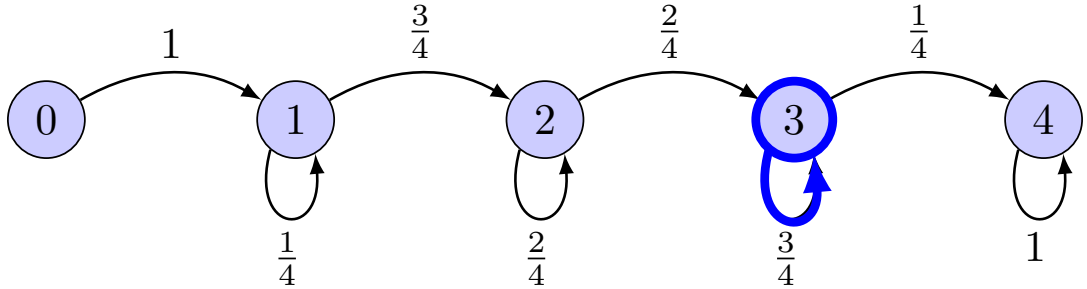
Viewing our first simulation run as a Markov chain …



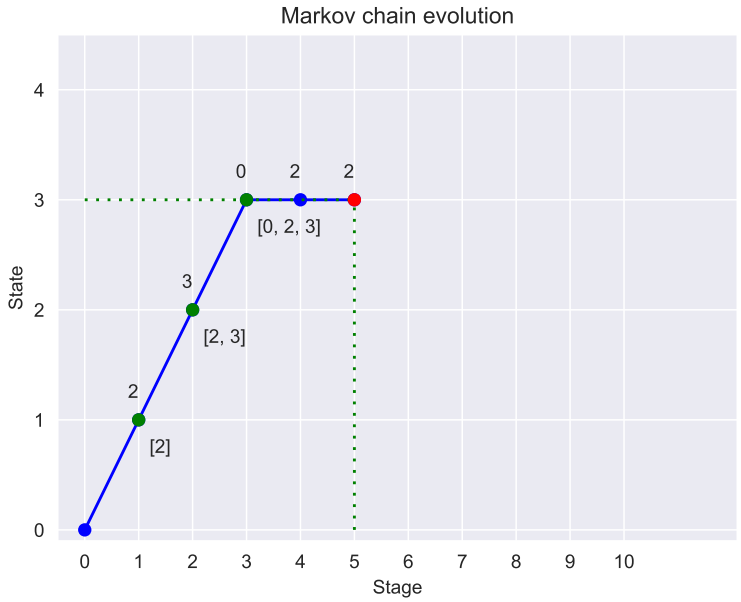

```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
```



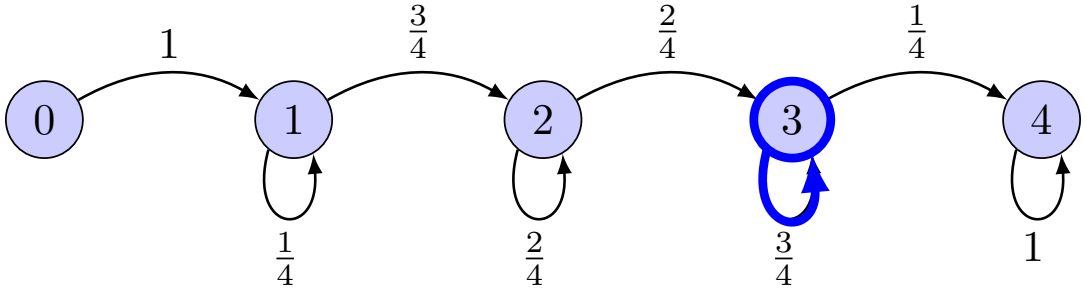
Viewing our first simulation run as a Markov chain …



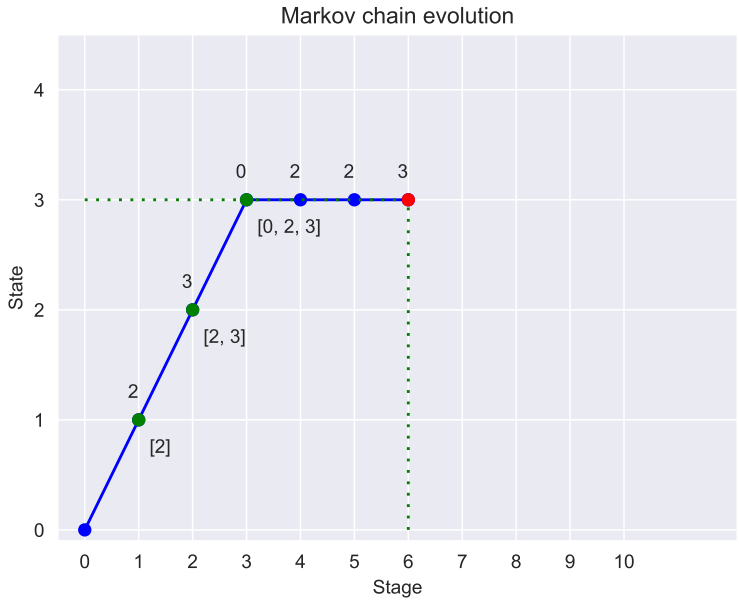

```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
stage =   5   found = 2   collected = {0, 2, 3}
```



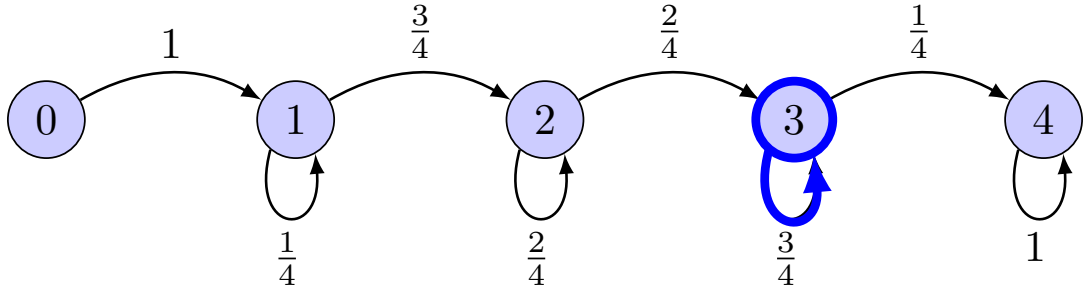
Viewing our first simulation run as a Markov chain …



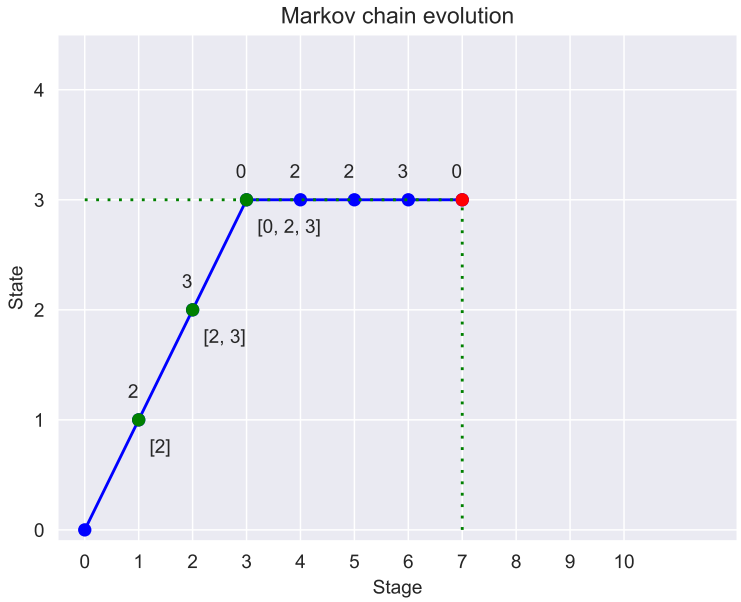

```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
stage =   5   found = 2   collected = {0, 2, 3}
stage =   6   found = 3   collected = {0, 2, 3}
```



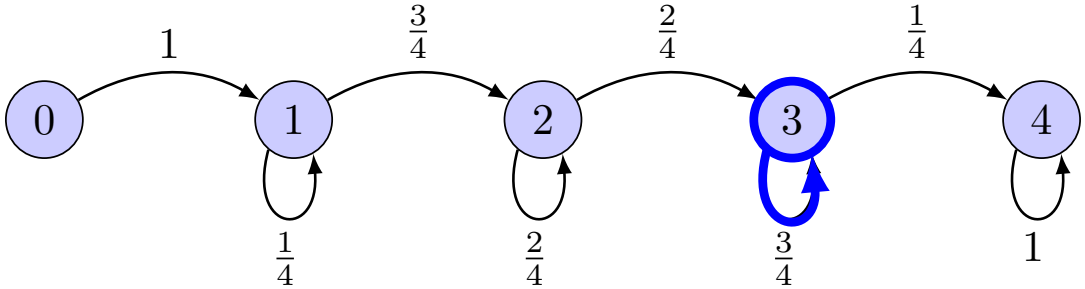
Viewing our first simulation run as a Markov chain …

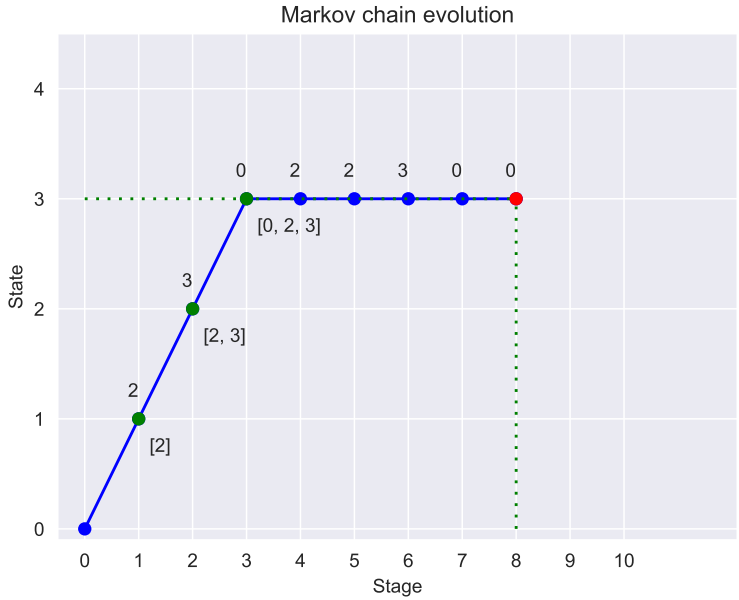



```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
stage =   5   found = 2   collected = {0, 2, 3}
stage =   6   found = 3   collected = {0, 2, 3}
stage =   7   found = 0   collected = {0, 2, 3}
```



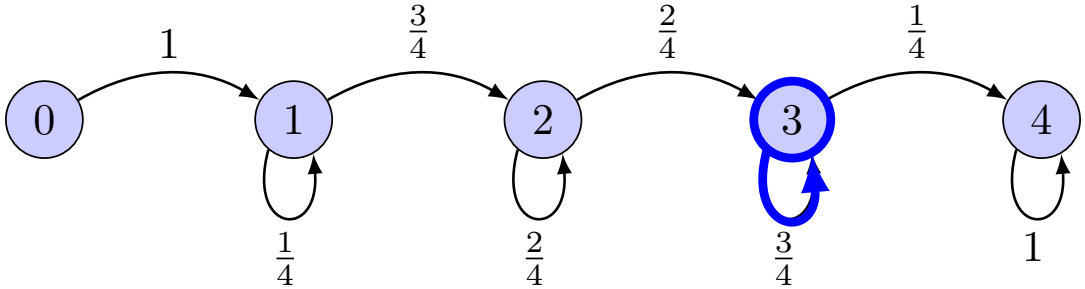
Viewing our first simulation run as a Markov chain …

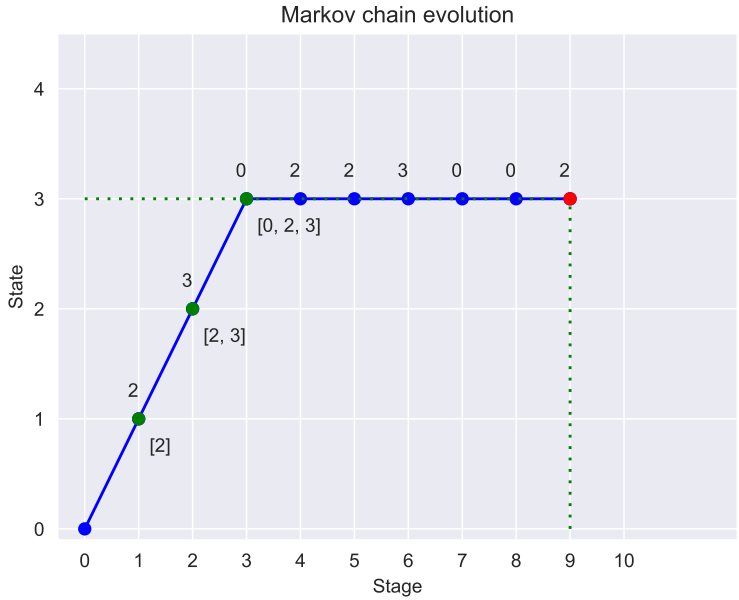





```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
stage =   5   found = 2   collected = {0, 2, 3}
stage =   6   found = 3   collected = {0, 2, 3}
stage =   7   found = 0   collected = {0, 2, 3}
stage =   8   found = 0   collected = {0, 2, 3}
```

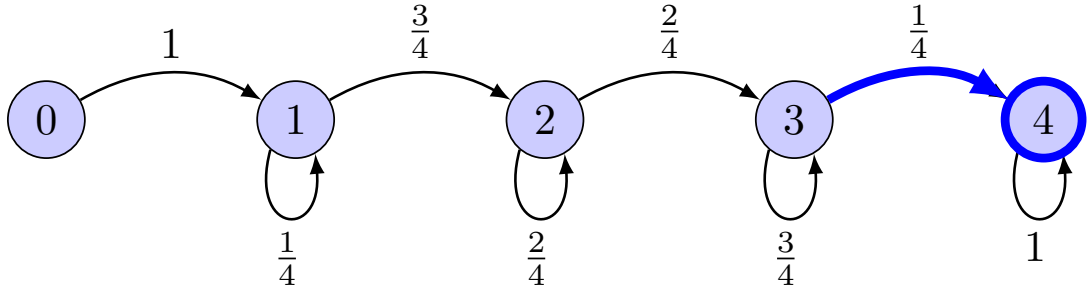
Viewing our first simulation run as a Markov chain …



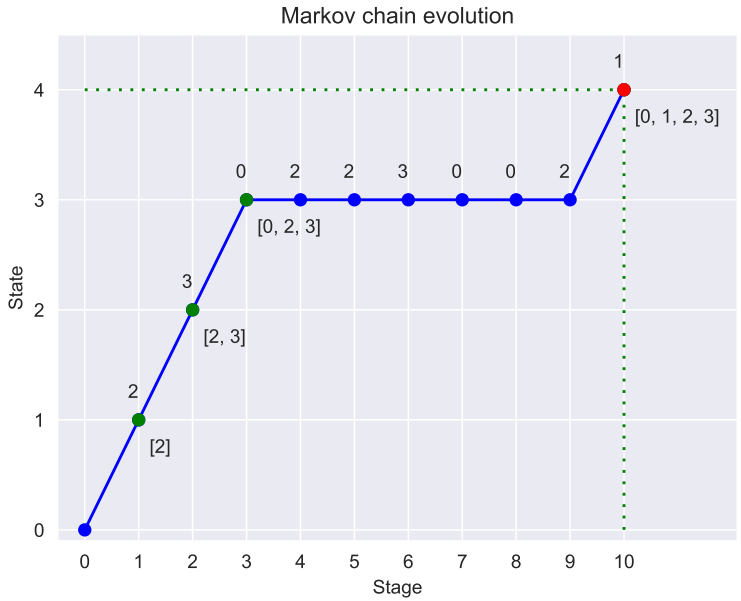



```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
stage =   5   found = 2   collected = {0, 2, 3}
stage =   6   found = 3   collected = {0, 2, 3}
stage =   7   found = 0   collected = {0, 2, 3}
stage =   8   found = 0   collected = {0, 2, 3}
stage =   9   found = 2   collected = {0, 2, 3}
```

Viewing our first simulation run as a Markov chain …




```
stage =   0   found = None  collected = set()
stage =   1   found = 2   collected = {2}
stage =   2   found = 3   collected = {2, 3}
stage =   3   found = 0   collected = {0, 2, 3}
stage =   4   found = 2   collected = {0, 2, 3}
stage =   5   found = 2   collected = {0, 2, 3}
stage =   6   found = 3   collected = {0, 2, 3}
stage =   7   found = 0   collected = {0, 2, 3}
stage =   8   found = 0   collected = {0, 2, 3}
stage =   9   found = 2   collected = {0, 2, 3}
stage =  10   found = 1   collected = {0, 1, 2, 3}
```



Number of stages needed = 11

Tasks

We now have

- a simulation of the coupon collector problem,
- a theoretical approach to calculating the expected number of coupons needed, and
- a Markov chain model of the problem.

We now want to look at extensions of the basic problem, e.g., non-uniform probabilities, multiple collections, etc.

Prep Work … be clear on your starting point.

Simulation

We have function `run_experiment(n, seed, debug)`.

But is it fast?
Asking gemini to review … it starts by praising me[1] "Your code is clean, explicit, and perfectly traces the exact logic … is a fantastic baseline …" but then "… this implementation contains a few computational bottlenecks typical of pure Python code."

Gemini gave suggestions for improving the code, with claim of 100x-200x improvement over the original code[2]. **You should read the suggestions and implement the ones that you understand**. Later you will be asked to modify this code during our F2F sessions so your understanding is critical, speed is a distant second.

Markov Chain Model

Implement function `transition_matrix(n)` that returns the transition matrix for the Markov chain model of the coupon collector problem with n coupons.

[1] We, old white guys, love being praised - see Richard Dawkins and The Claude Delusion

[2] The actually speed up was dependent on n, with 12x for $n = 100$, 132x for $n = 1000$, and 201x for $n = 2000$.

The Non-Uniform Coupon Collector

Problem

In the basic problem, every coupon has equal probability ($1/n$) of being found. In many real-life situations, this is not the case.

Probability of coupon i is p_i. for $i = 1, \ldots, n$.

Application

Taking motivation from physics, we assume coupon i has probability $p_i \propto e^{-\beta E_i}$ (Boltzmann distribution). This simulates a particle trying to visit all available energy states via thermal fluctuations, where

- $\beta = 1/(k_B T)$ is the inverse temperature,
- E_i is the energy of state i, given by (one of)
  - $E_i = (i-1) \times \varepsilon_0$ (Linear Energy Ladder - Quantum Oscillator Model)
  - $E_i = \ln(i)$ (The Zipf's Law Model)

Of interest is how does the expected number of coupons needed to complete the collection change as we vary β, in particular as $\beta \to 0$?

The Non-Uniform Coupon Collector

Theory

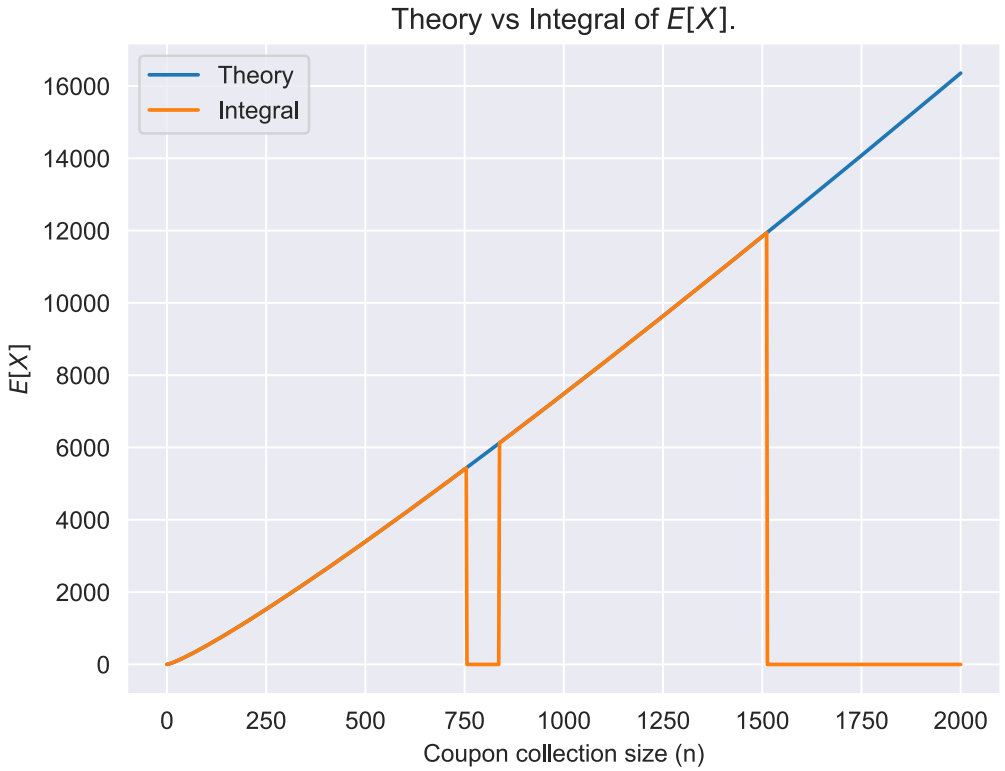
The expected number of coupons needed to complete the collection is given by [1] (Eq. 13b)

$$\mathrm{E}[X] = \int_0^\infty \left(1 - \prod_{i=1}^{n} (1 - e^{-p_i t})\right) \mathrm{d}t$$

Questions

- Verify that the general integral formula for the expected number of coupons needed to complete the collection reduces to the known formula for the uniform case when $p_i = \frac{1}{n}$ for all i.
- Which of the approaches (simulation or Markov chain) is best for this problem?
- Validate your implementation by comparing it to the integral formula.
- How does the expected number of coupons needed to complete the collection change as we vary β? In particular, what happens as $\beta \to 0$?

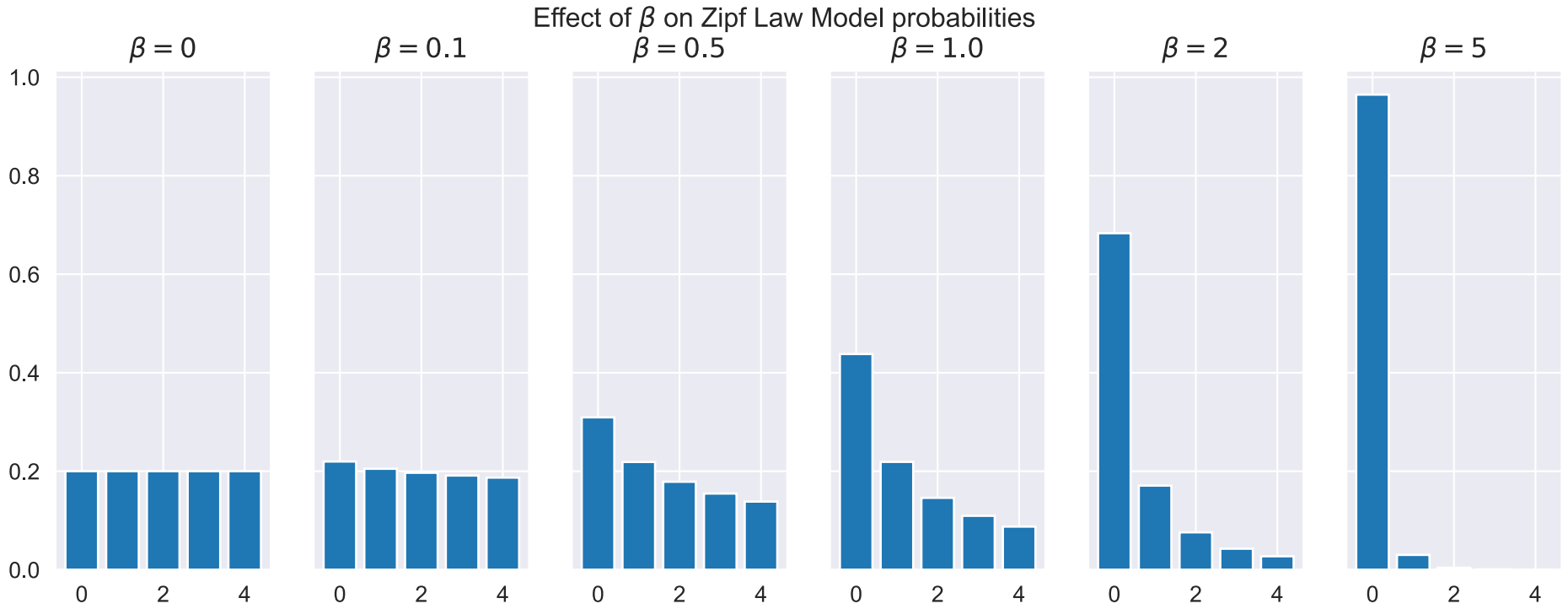
Theory vs Integral (Uniform case)



What is happening here ?

- Integrand involves exponentials which rapidly decay for large n in the uniform case (and small p in the general case).
  - The `scipy` function used, `integrate.quad`, is unable to converge in this situation and returns -1.
  - Cannot compute for $n > 1,500$, but see failure interval at half that value.

Effect of β on Zipf Law Model Probabilities



- As $\beta \to 0$ probabilities tend towards uniform probability case.
- As β increases the probability distribution tends towards $[1, 0, 0, \ldots, 0]$ and the time needed to collect all coupons tends towards infinity.

Effect of β on Expected Time to Collect all Coupons



- As $\beta \to 0$ probabilities tend towards uniform probability case. Here, since $n = 5$, the limit is 11.4.
- As β increases time needed to collect all coupons tends towards infinity.

The “Collectible Pairs” Problem

Problem

Instead of collecting just one of each coupon, we must collect at least m copies of every single coupon (e.g., $m = 2$ to make matching pairs).

This mirrors physical processes like nucleation, where a crystal or droplet won’t grow stably until a local cluster reaches a critical threshold minimum of atoms, rather than just a single random arrival.

The “Leaky Pocket” Collector

Problem

This version introduce a loss mechanism. Every time a collector buys a new coupon, there is a small probability, q, that they accidentally lose one random coupon they’ve already collected (e.g., it drops out of a leaky pocket).

Questions

- As q tends to zero, the problem converges to the basic problem.
- As q increases the collector starts losing coupon at a rate faster than they are collecting more so, at some point, so the expected time becomes infinite because the system reaches a dynamic steady-state equilibrium where the rate of gaining new coupons matches the rate of losing them.

The Coupon Collector with “Trading”

Problem

This is an agent-based model. Instead of one isolated collector, simulate a network of M collectors who all start collecting simultaneously. When they draw a duplicate coupon they don’t need, they can trade it with a peer.

References

References

[1] “Birthday paradox, coupon collectors, caching algorithms and self-organizing search.” [Online]. Available: https://www.sciencedirect.com/science/article/pii/0166218X9290177C